



Choix du projet et répartition des rôles

SAE2 JUPITER4 — Suivi de Colis

Semestre 4 — 2025/2026
BUT Informatique — Université Sorbonne Paris Nord
IUT de Villetaneuse

Membre	Rôle
Amir TABELLOUT	Chef de projet, architecture technique, front-end, interface Agent
Boran CAV	Interface Service Financier, authentification
Anas LACENE-NECER	Interface Directeur, base de données
Amira BENHADDI	Interface CRIT, système de permissions



IUT Villetaneuse
Université Sorbonne Paris Nord

1. Pourquoi on a choisi le projet Jupiter B

Quand on a dû choisir un projet à reprendre pour la SAE2, on avait plusieurs options. Le sujet était le même pour tout le monde (suivi de colis pour l'IUT), mais chaque groupe de l'année précédente avait fait sa propre version, avec des stacks et des niveaux d'avancement différents.

On a regardé le code de Jupiter B et c'est celui qui nous a semblé le plus intéressant à reprendre. Pas parce qu'il était le plus avancé, au contraire. Mais parce que la base était propre.

Concrètement, le projet utilisait Laravel avec une architecture MVC correcte. Les modèles Eloquent étaient bien définis : User, Order, Role, Supplier, Permission, Package, Comment, Log. Les relations étaient en place. Les migrations créaient les bonnes tables. Il y avait un système de rôles et de permissions en base de données avec des tables pivot. Ça, c'est le genre de fondation qu'on ne voulait pas réécrire de zéro. De plus, le CAS est déjà intégré, c'est pile ce qui nous manquait pour notre SAE précédente.

Par contre, il manquait à peu près tout ce qui est visible. Pas de tableaux de bord, pas de wizard de création de commande, pas d'interface pour le service financier ni pour le directeur, rien pour le CRIT, un design inconsistant, pas de génération de PDF, pas d'envoi d'emails. Le projet avait les os mais pas la chair.

C'est exactement ce qu'on cherchait. Repartir d'une base saine nous évitait de perdre du temps sur le schéma de base de données et la structure MVC, et nous laissait le champ libre pour construire toutes les interfaces et les fonctionnalités. Ça voulait aussi dire qu'on avait beaucoup de travail à montrer au jury, parce que la quasi-totalité de ce qui est visible à l'écran a été écrite par nous.

Un autre argument : Laravel.

Lors de la SAE1, Amir qui gère le Front-end a fait celui-ci en HTML/CSS/JS et PHP brut. Ça fonctionnait, mais c'était difficile à maintenir. Reprendre un projet Laravel permettait d'apprendre un vrai framework avec un ORM, un moteur de templates, des migrations, un système de mail intégré. En termes de montée en compétences, c'était plus formateur que de refaire du PHP natif.

Le projet avait aussi une configuration Docker fonctionnelle. On pouvait cloner le dépôt, lancer docker compose up, et avoir l'application qui tourne. Pour un projet d'équipe, c'est un vrai avantage : tout le monde travaille dans le même environnement, on évite les « ça marche sur mon PC ».

Ce qui a fait pencher la balance

Finalement, le choix s'est fait sur un critère simple : on voulait un projet où on pouvait apporter un maximum de valeur ajoutée. Jupiter B avait le bon équilibre entre une fondation technique solide et un manque de fonctionnalités suffisant pour qu'on puisse montrer un travail conséquent. Les autres projets étaient soit trop avancés (peu de marge pour améliorer), soit trop instables (il aurait fallu tout reprendre, y compris la base de données).

2. Répartition des rôles et des tâches

On s'est réparti le travail par interface. Chaque membre est responsable d'une interface utilisateur complète (front + back) et d'un aspect technique transversal. L'idée c'est que chacun maîtrise son périmètre de bout en bout et puisse le présenter en autonomie devant le jury.

Amir TABELLOUT

Chef de projet, architecture technique, front-end, interface Agent

Amir coordonne le projet. C'est lui qui a défini l'architecture globale de l'application, la structure des routes, et la façon dont les contrôleurs communiquent avec les vues.

Tâche	Détail
Design system	Variables CSS (couleurs, rayons, ombres), composants visuels (cartes KPI, badges, tableaux), layout responsive avec sidebar + topbar
Template Blade base	Fichier base.blade.php dont toutes les pages héritent. Contient la sidebar, la topbar, le conteneur de modale, et le JavaScript des modales AJAX
Interface Agent	Dashboard agent avec KPIs département, historique des commandes, export CSV
Wizard de création	Formulaire en 3 étapes avec persistance en session Laravel : infos générales, fournisseur + articles avec calcul TVA dynamique, récapitulatif
Intégration front	Intégration visuelle de toutes les interfaces des autres membres pour garder la cohérence
Dashboard SuperAdmin	Vue d'ensemble système avec KPIs globaux, gestion des utilisateurs, logs d'audit, statistiques

Amir avait déjà fait la SAE1 sur le même sujet, donc il connaissait bien les besoins fonctionnels. Ça l'a aidé à définir rapidement ce qu'il fallait construire et dans quel ordre.

Boran CAV

Interface Service Financier, authentification

Boran a pris en charge la partie la plus dense du workflow : le service financier. C'est le rôle qui intervient le plus souvent dans le cycle de vie d'une commande (validation, envoi, paiement). Il a aussi géré le système d'authentification.

Tâche	Détail
Dashboard SF	KPIs financiers (devis en attente, BC à envoyer, paiements), actions rapides
Validation des devis	Page listant les devis soumis, modale AJAX de validation avec upload du bon de commande en PDF
Envoi de BC	Envoi d'email au fournisseur avec le BC signé en pièce jointe via le système Mail de Laravel
Relance fournisseur	Template d'email de relance, modale avec message personnalisable
Paiement	Confirmation de paiement, passage au statut terminal LIVRÉ_PAYÉ
Gestion fournisseurs	CRUD complet : ajout, modification, suspension, détails avec statistiques
Auth CAS	Configuration du protocole CAS pour l'authentification universitaire en production
Page dev-login	Interface de connexion rapide en développement, cartes cliquables par rôle, filtrage, badges colorés
Protection CSRF	Tokens CSRF sur les formulaires et les requêtes AJAX, balise meta csrf-token

Anas LACENE-NECER

Interface Directeur, base de données

Anas s'est occupé de l'interface du directeur et de tout ce qui touche à la base de données. La signature électronique est la fonctionnalité la plus technique de son périmètre.

Tâche	Détail
Dashboard Directeur	KPIs de signature, compteur de BC en attente, montants concernés
Signature électronique	Canvas HTML5 pour dessiner la signature dans le navigateur, ou upload d'image. Conversion en base64 via <code>toDataURL()</code> , envoi au serveur
Génération PDF signé	Utilisation de DomPDF pour générer le PDF du bon de commande avec la signature intégrée. Résolution du problème d'images en base64 inline
Refus de BC	Modale de refus avec motif obligatoire, retour au SF pour correction
Historique signatures	Page listant toutes les signatures passées avec détails, export CSV
Migrations Laravel	Les 17 fichiers de migration qui structurent les 13 tables de la base de données
Schéma relationnel	Tables users, orders, articles, roles, permissions, suppliers, packages, comments, logs, avec les pivots <code>role_user</code> et <code>permission_role</code>
Conversion ENUM→VARCHAR	Migration du type de la colonne status : passage de ENUM à VARCHAR pour contourner les différences MariaDB/MySQL
DatabaseSeeder	Création automatique des 10 rôles, 17 permissions, 5 comptes de test, 3 fournisseurs, avec les associations pivot

Amira BENHADDI

Interface CRIT, système de permissions

Amira a construit l'interface du service courrier (CRIT), qui gère la réception et la distribution des colis. Elle a aussi conçu le système de permissions qui contrôle l'accès à chaque fonctionnalité.

Tâche	Détail
Dashboard CRIT	KPIs réception/distribution, colis en attente
Réception de colis	Page listant les commandes en attente de réception. Modale de saisie du réceptionnaire, confirmation de réception
Livraison partielle	Gestion du cas où tous les colis ne sont pas arrivés : statut PARTIEL, la commande réapparaît dans la file
Distribution	Confirmation de livraison complète au département, passage en statut SERVICE_FAIT
Historique CRIT	Page de consultation des opérations passées, filtrage par date et statut
Export CSV	Génération serveur du fichier CSV avec en-têtes et données, réponse avec Content-Type text/csv
Enum PermissionValue	Définition des 17 permissions dans un enum PHP : ADMIN, CONSULTER_TOUTES_COMMANDES, CRÉER_COMMANDES, SIGNER_BONS_DE_COMMANDES, etc.
Associations permission-rôle	Configuration des tables pivot définissant quelles permissions vont à quel rôle
Méthode hasPermission()	Implémentation dans le modèle User : charge les rôles, récupère les permissions via les relations Eloquent, retourne un booléen

3. Vue d'ensemble

Membre	Interface	Aspect technique	Pages créées
Amir TABELLOUT	Agent + SuperAdmin	Architecture, front-end, design system	Dashboard Agent, Wizard 3 étapes, Historique Agent, Dashboard Admin, Users, Logs, Stats
Boran CAV	Service Financier	Authentification (CAS + dev-login)	Dashboard SF, Validation devis, Suivi commandes, Gestion fournisseurs
Anas LACENE-NECER	Directeur	Base de données, migrations, seeder	Dashboard Directeur, Signature BC, Historique signatures
Amira BENHADDI	CRIT	Permissions + rôles	Dashboard CRIT, Réception, Distribution, Historique CRIT

Chaque membre présente son interface lors de la démonstration. On fait un parcours complet d'une commande en changeant de rôle à chaque étape : Amir crée la commande en tant qu'agent, Boran la valide en tant que SF, Anas la signe en tant que directeur, Boran l'envoie au fournisseur, Amira reçoit et distribue le colis en tant que CRIT, et Boran confirme le paiement.

Les aspects techniques transversaux (auth, BDD, permissions, front-end) sont présentés par le membre qui les a implémentés, intégrés dans la partie de présentation de son interface.